# THỰC NGHIỆM GIÁ TRỊ CUTOFF TỐI ƯU CHO MERGESORT


## Tóm tắt

Mergesort thường được cải tiến bằng cách chuyển sang insertion sort khi mảng con đủ ngắn; giáo trình nêu ngưỡng CUTOFF là 7 (phiên bản cũ) và khoảng 10 (phiên bản mới). Báo cáo này xác định bằng thực nghiệm ngưỡng phù hợp cho cài đặt mergesort top-down dựng từ bài giảng, chạy trên Java 25 với máy AMD Ryzen AI 7 H 350. Hai phiên bản được đo: V1 chỉ có cutoff, V2 có đủ ba cải tiến (cutoff, bỏ merge khi đã có thứ tự, bỏ sao chép bằng đổi vai trò mảng). Mỗi phiên bản được đo với 51 giá trị cutoff (0–40 và 44–80 bước 4), sáu kích thước $n$ từ $10^3$ đến $10^6$, năm phân phối dữ liệu, 30 trial; thiết kế ghép cặp (mọi cutoff dùng chung một mảng trong mỗi trial) và kiểm định Wilcoxon có hiệu chỉnh Holm. Trên dữ liệu ngẫu nhiên, cutoff tốt nhất giúp V1 nhanh hơn 10,3–27,0% và V2 nhanh hơn 3,4–14,4%. Do mergesort chia đôi, thời gian là hàm bậc thang của CUTOFF và chỉ phụ thuộc vào độ dài lá thực tế; vì vậy $L^*$ được hiểu là một vùng. Tiêu chí chọn một giá trị dùng chung cho mọi $n$ (cực tiểu mức chậm hơn tối đa so với tối ưu) cho CUTOFF ≈ 26 với V1 (chậm hơn tối ưu tối đa 3,6%) và CUTOFF = 10 với V2 (tối đa 2,7%). Giữa 7 và 10: 10 nhanh hơn có ý nghĩa ở 5/12 trường hợp, không trường hợp nào 7 nhanh hơn. $L^*$ phụ thuộc vào hệ thống, vào $n$ (qua cấu trúc lá và trạng thái cache) và vào phiên bản: bỏ bước sao chép làm $L^*$ của V2 nhỏ hơn rõ rệt so với V1.


## 1 Giới thiệu

Mergesort top-down có số phép so sánh $\sim N \lg N$ trong mọi trường hợp, nhưng với mảng con rất ngắn thì chi phí phụ của đệ quy và của bước merge (sao chép sang mảng phụ, nhiều nhánh rẽ) lại lớn hơn công việc thực sự. Cải tiến phổ biến là *cutoff*: mảng con có độ dài không vượt quá một ngưỡng thì sắp xếp bằng insertion sort (**??**). `Arrays.sort` cho đối tượng trong Java 6 và `MergeX.java` của giáo trình dùng ngưỡng 7; bài giảng mới nêu giá trị khoảng 10. Tuy nhiên ngưỡng tốt nhất phụ thuộc vào phần cứng, JVM và chi tiết cài đặt, nên cần được đo.

Báo cáo trả lời ba câu hỏi nghiên cứu:

- **RQ1.** Trên máy và cài đặt của tôi, CUTOFF nào cho thời gian nhỏ nhất?
- **RQ2.** Giữa 7 và 10, giá trị nào tốt hơn?
- **RQ3.** $L^*$ có phụ thuộc vào $n$, phân phối dữ liệu và phiên bản cài đặt không?

Đóng góp chính:

- Hai cài đặt mergesort (V1, V2) dựng từ bài giảng, được kiểm thử bằng 112 680 ca test và 5 lỗi cài cố ý (mutation testing), tất cả đều bị phát hiện.
- Một harness đo có thể tái lập (seed, checksum SHA-256, manifest), thiết kế ghép cặp, đã kiểm tra nhiễu nền (A/A test), hiệu ứng thứ tự và GC.
- Chỉ ra rằng thời gian là hàm bậc thang của CUTOFF; các cutoff cùng "lớp lá" là cùng một thuật toán, dùng làm đối chứng âm cho kiểm định thống kê.
- Khuyến nghị cụ thể cho máy này: V1 dùng CUTOFF ≈ 24–29, V2 dùng 10–14.

Mục 2 trình bày cơ sở lý thuyết, Mục 3 cài đặt, Mục 4 thiết kế thí nghiệm, Mục 5 kết quả, Mục 6 thảo luận, Mục 7 kết luận và Mục 8 nêu các hạn chế.

## 2 Cơ sở lý thuyết

### 2.1 Mergesort top-down và ba cải tiến

Mergesort top-down chia mảng $a[lo..hi]$ tại $mid = lo + \lfloor (hi - lo)/2 \rfloor$, sắp xếp đệ quy hai nửa rồi trộn (merge) chúng qua mảng phụ `aux`. Bước merge sao chép $a[lo..hi]$ sang `aux`, sau đó lần lượt lấy phần tử nhỏ hơn ở

đầu hai nửa; khi bằng nhau thì lấy nửa trái để giữ tính ổn định. Thuật toán dùng tối đa $N \lg N$ phép so sánh và $6N \lg N$ lượt truy cập mảng.

Ba cải tiến trong bài giảng: (1) *cutoff*: mảng con ngắn được sắp xếp bằng insertion sort; (2) *dừng nếu đã có thứ tự*: nếu $a[mid] \le a[mid+1]$ thì bỏ qua merge; (3) *bỏ sao chép*: đổi vai trò `a` và `aux` ở mỗi tầng đệ quy để không phải chép sang `aux` trước mỗi lần merge.

Trong báo cáo này, một mảng con là *lá* khi `hi <= lo + CUTOFF - 1`, tức là độ dài ≤ CUTOFF (theo slide). Định nghĩa này khác `MergeX.java` (điều kiện `hi <= lo + CUTOFF`, tức độ dài ≤ CUTOFF+1), nên "CUTOFF 7" của MergeX tương ứng CUTOFF 8 ở đây.

### 2.2 Insertion sort

Insertion sort trên $N$ phần tử khác nhau dùng trung bình $\sim N^2/4$ phép so sánh và $\sim N^2/4$ phép đổi chỗ; tốt nhất (đã có thứ tự) $N-1$ phép so sánh và không đổi chỗ; xấu nhất (thứ tự ngược) $\sim N^2/2$ phép so sánh và đổi chỗ. Dù có bậc $N^2$, với $N$ nhỏ insertion sort vẫn nhanh: vòng lặp ngắn và chặt, không gọi đệ quy, không cần mảng phụ, truy cập các ô nhớ liền kề (luôn nằm trong cache L1) và nhánh thoát vòng lặp trong chỉ bị đoán sai khoảng một lần mỗi lượt chèn.

### 2.3 Mô hình chi phí và giá trị tối ưu

Gọi $L$ là độ dài lá. Mô hình chi phí được dùng là

$$T(N, L) \approx a \frac{NL}{4} + b\, N\, (\lg N - \lg L) \tag{1}$$

Số hạng thứ nhất là phần insertion sort: có $N/L$ lá, mỗi lá tốn $\sim L^2/4$ phép so sánh, nên tổng là $NL/4$; hệ số $a$ là chi phí (ns) của một bước trong vòng lặp insertion sort (một so sánh và một đổi chỗ). Số hạng thứ hai là phần merge: cây đệ quy còn $\lg N - \lg L$ tầng, mỗi tầng xử lý $N$ phần tử; $b$ là chi phí trên mỗi phần tử ở mỗi tầng (so sánh, sao chép sang `aux`, ghi lại, gọi đệ quy).

Đạo hàm theo $L$: $\partial T/\partial L = aN/4 - bN/(L \ln 2) = 0$, suy ra

$$L^* \approx \frac{4b}{a \ln 2}. \tag{2}$$

Ba hệ quả: (i) $L^*$ chỉ phụ thuộc tỉ số $b/a$, tức phụ thuộc phần cứng, JVM và cách cài đặt merge (V1 có thêm bước sao chép nên $b$ lớn hơn V2, dự đoán $L^*(\text{V1}) > L^*(\text{V2})$); (ii) $N$ bị triệt tiêu, nên mô hình dự đoán $L^*$ không phụ thuộc $N$; (iii) gần cực tiểu $T$ thay đổi chậm (đạo hàm bằng 0), nên đường cong phẳng và có cả một vùng cutoff cho thời gian gần như nhau.

### 2.4 Hiệu ứng bậc thang

Vì mergesort top-down luôn chia đôi, ở mỗi tầng của cây đệ quy chỉ có một hoặc hai độ dài mảng con ($\lfloor m/2 \rfloor$ và $\lfloor m/2 \rfloor$). Khi tăng CUTOFF, cây đệ quy chỉ thay đổi khi CUTOFF vượt qua một trong các độ dài đó; giữa hai mốc liên tiếp, mọi CUTOFF sinh ra *cùng một cây*, tức cùng một thuật toán. Do đó $T(L)$ là hàm bậc thang. Ta gọi tập độ dài lá thực tế là *lớp lá* của CUTOFF. Bảng 1 liệt kê các độ dài có thể là lá; chẳng hạn với $n = 100\,000$, mọi CUTOFF từ 49 đến 96 đều cho lá $\{48, 49\}$. Vì $N$ và $2N$ có cùng các độ dài này (một tầng nhiều hơn), $n = 100\,000$ và $n = 200\,000$ có cùng cấu trúc lá.

| $n$ | Độ dài lá có thể có |
|---|---|
| 1 000 | 1 2 3 4 7 8 15 16 31 32 62 63 125 |
| 10 000 | 1 2 3 4 5 9 10 19 20 39 40 78 79 |
| 100 000 | 1 2 3 4 6 7 12 13 24 25 48 49 97 98 |
| 200 000 | 1 2 3 4 6 7 12 13 24 25 48 49 97 98 |
| 300 000 | 1 2 3 4 5 9 10 18 19 36 37 73 74 |
| 700 000 | 1 2 3 5 6 10 11 21 22 42 43 85 86 |
| 1 000 000 | 1 2 3 4 7 8 15 16 30 31 61 62 122 123 |

Bảng 1: Các độ dài mảng con có thể là lá (≤ 128) cho từng $n$ (analysis/out/leaf_table.csv)

### 2.5 Phương pháp thực nghiệm

Thí nghiệm theo chu trình của phương pháp khoa học trong bài giảng tuần 2: quan sát, đặt giả thuyết, đưa ra dự đoán, kiểm chứng bằng đo đạc và xác nhận lại bằng một thí nghiệm độc lập (TN3). Thí nghiệm phải tái lập được (mã nguồn, seed, checksum, tham số JVM đều được ghi lại) và giả thuyết phải bác bỏ được (dự đoán và quy tắc ra quyết định được ghi trước khi đo). Thời gian chạy chịu ảnh hưởng của các yếu tố phụ thuộc hệ thống: phần cứng (bộ nhớ đệm, lõi không đồng nhất, tần số xung nhịp), phần mềm (trình biên dịch JIT, bộ thu gom rác) và hệ thống (ứng dụng chạy nền, bộ lập lịch của hệ điều hành). Mục 4.4 mô tả biện pháp xử lý từng yếu tố.

## 3 Cài đặt

### 3.1 V1: Mergesort với cutoff

V1 (`MergeCutOff`) là mergesort top-down của bài giảng, nhận `cutoff` làm tham số (không phải hằng số) để cùng một mã đo được mọi giá trị:

```
void sort(Comparable[] a, Comparable[] aux,
        int lo, int hi, int cutoff) {
  if (hi <= lo) return;
  if (hi <= lo + cutoff - 1) {
    InsertionSort.sort(a, lo, hi); return; }
  int mid = lo + (hi - lo) / 2;
  sort(a, aux, lo, mid, cutoff);
  sort(a, aux, mid+1, hi, cutoff);
  merge(a, aux, lo, mid, hi); // chép sang aux rồi trộn
}
```

Phép thử `hi <= lo` đặt trước phép thử cutoff, nên cutoff 0 và cutoff 1 đều cho đúng mergesort gốc (lá độ dài 1). Hai cấu hình này là cùng một thuật toán và được dùng làm A/A test để đo nhiễu nền. Insertion sort theo đoạn dùng điều kiện `j > lo` để không chạm vào phần tử ngoài [*lo*, *hi*]; nó dùng phép đổi chỗ `exch` như trong bài giảng. Cutoff âm ném `IllegalArgumentException`.

### 3.2 V2: đủ ba cải tiến

V2 (`MergeOptimized`) theo `MergeX` của giáo trình. Bất biến quan trọng nhất: *sau lời gọi `sort(src, dst, lo, hi)`, đoạn* [*lo*, *hi*] *đã có thứ tự nằm trong `dst`.*

```
void sort(Comparable[] a, int cutoff) {
  Comparable[] aux = a.clone();
  sort(aux, a, 0, a.length - 1, cutoff); }
void sort(Comparable[] src, Comparable[] dst,
        int lo, int hi, int cutoff) {
  if (hi <= lo) return;
  if (hi <= lo + cutoff - 1) {
    InsertionSort.sort(dst, lo, hi); return; }
  int mid = lo + (hi - lo) / 2;
  sort(dst, src, lo, mid, cutoff);
  sort(dst, src, mid+1, hi, cutoff);
  if (!less(src[mid+1], src[mid])) {
    System.arraycopy(src, lo, dst, lo, hi-lo+1);
    return; }
  merge(src, dst, lo, mid, hi); }
```

Các điểm thiết kế: (a) hai lời gọi con đổi vai trò `sort(dst, src, ...)` nên hai nửa đã có thứ tự nằm trong `src`, rồi merge từ `src` sang `dst` mà không cần sao chép trước; (b) `aux = a.clone()` trước khi đệ quy để hai mảng có cùng nội dung ban đầu: ở một lá, đoạn [*lo*, *hi*] của cả hai mảng chưa bị lời gọi nào ghi vào, nên insertion sort trực tiếp trên `dst` là đúng, và lá độ dài 1 trả về ngay mà `dst[lo]` vẫn đúng; (c) phép thử đã-có-thứ-tự kiểm tra trên `src` (nơi hai nửa đang nằm), và khi bỏ qua merge vẫn phải `arraycopy` từ `src` sang `dst` để giữ bất biến; (d) ở mức gọi ngoài cùng, `dst` là mảng `a` của người dùng, nên kết quả nằm đúng chỗ.

### 3.3 Kiểm thử tính đúng

`SortTest` gồm bốn nhóm, chạy trên 229 kích thước (0–200, $2^k - 1$, $2^k$, $2^k + 1$ đến $2^{16}$, và 100 000), mọi cutoff 0–40 và sáu phân phối (RANDOM, FEW_KEYS, SORTED, REVERSED, ALL_EQUAL, SAWTOOTH):

- **A** (56 334 ca mỗi phiên bản): kết quả trùng `Arrays.sort`.
- **B** (4 000 ca): `InsertionSort.sort(a, lo, hi)` chỉ sắp xếp và chỉ chạm vào $[lo, hi]$.
- **C** (56 334 ca mỗi phiên bản): tính ổn định (khoá bằng nhau giữ thứ tự ban đầu).
- **D** (12 ca): cutoff âm phải ném `IllegalArgumentException`.

SAWTOOTH (các khúc tăng dần có độ dài ngẫu nhiên) được đưa vào vì nó kích hoạt nhánh "bỏ qua merge" của V2 ở một số nút nhưng không phải tất cả, là chỗ dễ sai nhất của V2. Kích thước $2^k \pm 1$ kiểm tra các trường hợp chia lệch. Tổng cộng 112 680 ca test qua trong khoảng 36 s; script chạy thí nghiệm dừng nếu `SortTest` thất bại. Để kiểm tra chính bộ test, năm lỗi được cài cố ý (Bảng 2), mỗi lỗi chạy lại toàn bộ bốn nhóm; cả năm đều bị phát hiện.

| Lỗi cài cố ý | Kết quả |
|---|---|
| M1: `j > lo` → `j > 0` | B; A, C (V1, V2) |
| M2: lấy phần tử phải trước khi bằng nhau (V1) | chỉ C |
| M3: `mid+1` → `mid` (lời gọi đệ quy, V1) | A, C (StackOverflowError) |
| M4: bỏ `if (hi <= lo) return;` (V1) | A, C (StackOverflowError, cutoff 0) |
| M5 (V2): bỏ `arraycopy` khi đã sắp xếp | A, C |

Bảng 2: Mutation testing: lỗi cài cố ý và nhóm test phát hiện

M2 chỉ bị nhóm C phát hiện vì `Integer` bằng nhau vẫn `equals` nhau, nên nhóm A không thấy được lỗi về tính ổn định. M4 chỉ lộ ra ở cutoff 0 (với cutoff $\geq 1$, phép thử cutoff đã chặn lá độ dài 1). Không lỗi nào bị nhóm D phát hiện, đúng như kỳ vọng vì D chỉ kiểm tra tham số.

## 4 Thiết kế thí nghiệm

### 4.1 Giả thuyết

Năm giả thuyết được ghi trước khi đo được trình bày trong Bảng 3.

| | Giả thuyết |
|---|---|
| H1 | Cutoff giúp nhanh hơn 10–15% |
| H2 | $L^*$ nằm trong 5–15 |
| H3 | $L^*$ không phụ thuộc $n$ |
| H4 | Đường cong phẳng quanh tối ưu |
| H5 | $L^*(\text{V2}) < L^*(\text{V1})$ |

Bảng 3: Các giả thuyết (ghi trước khi đo)

H1 và H2 dựa trên giáo trình; H3 và H4 là hệ quả (ii), (iii) của mô hình (1); H5 là hệ quả (i): V2 bỏ bước sao chép nên $b$ nhỏ hơn.

### 4.2 Các biến

**Biến độc lập:** CUTOFF (51 giá trị: 0–40 bước 1 và 44–80 bước 4); kích thước $n$; phân phối dữ liệu; phiên bản (V1, V2). **Biến phụ thuộc:** thời gian (ns) cho một lần sắp xếp, tính bằng tổng thời gian của một lô $k$ lần sắp xếp chia cho $k$. **Biến kiểm soát:** cùng máy, cùng JDK và cờ JVM, cùng kiểu phần tử (`Double` đóng hộp, so sánh qua `Comparable`), cùng mảng đầu vào cho mọi cutoff trong một trial, CPU được ghim vào một nhóm lõi.

### 4.3 Bộ dữ liệu

`DataGenerator` sinh năm phân phối giá trị trong $[0, 1)$: RANDOM (đều, `nextDouble`); SORTED và REVERSED (RANDOM rồi sắp xếp tăng / giảm); FEW_KEYS (10 giá trị khác nhau $k/10$, `FEW_KEYS_K` = 10); SAWTOOTH (các khúc liên tiếp độ dài 1000 được sắp tăng, `SAWTOOTH_RUN` = 1000). Seed của mỗi mảng chỉ phụ thuộc $(d, n, t)$: seed = mix64(mix64(mix64(BASE_SEED $\oplus$ $h(d)$) $\oplus$ $n$) $\oplus$ $t$), với $h(d)$ là `hashCode` của `d.name()` (không dùng `ordinal` để thêm phân phối mới không đổi seed cũ), `mix64` là hàm trộn SplitMix64, `BASE_SEED` = 20261108, sinh số bằng `java.util.Random` (**?**). Mỗi mảng có checksum SHA-256 trên dãy bit IEEE-754. `data/manifest.csv` ghi seed và checksum của 900 mảng (5 phân phối × 6 kích thước × 30 trial); 15 file mẫu (trial 0, $n \le 10^5$) được lưu trong `data/samples/`, phần còn lại sinh lại được bằng `DataGenerator`. Bộ sinh có tự kiểm tra: cùng seed cho cùng checksum, khác seed cho khác checksum, ghi–đọc file không đổi dữ liệu, đúng tính chất phân phối, không trùng seed. Seed và checksum của toàn bộ 300 mảng dùng trong TN1 và TN2 (ghi trong file kết quả) đều khớp với manifest.

### 4.4 Quy trình đo

Mỗi cặp (thí nghiệm, $n$, phân phối) chạy trong một JVM riêng để trạng thái JIT và heap không truyền sang cấu hình sau. Thứ tự các JVM được xáo bằng seed cố định để sự nóng dần của máy không trùng với thứ tự tăng của $n$.

Biện pháp cho từng yếu tố gây nhiễu: JIT chưa biên dịch xong → warm-up trên mảng không thuộc dữ liệu đo; độ phân giải đồng hồ ($\approx$ 100 ns) → đo theo lô $k$ lần; GC trong vùng đo → heap cố định 4 GB, Serial GC, `System.gc()` trước mỗi lô; trôi theo thời gian và hiệu ứng thứ tự → xáo thứ tự cấu hình trong mỗi trial; khác biệt giữa các mảng đầu vào → ghép cặp (mọi cấu hình dùng chung một mảng trong một trial); loại bỏ mã chết → cộng dồn một phần tử của kết quả vào `sink` và in ra; lõi CPU không đồng nhất → ghim JVM vào CPU logic 0–7. Dữ liệu được ghi xuống đĩa sau mỗi trial, kèm file `.meta.txt` (JVM, cờ, tham số, độ phân giải đồng hồ) và GC log.

```
warm-up: 10 vòng, n = 100000,
         seed trial âm, mọi cấu hình
         theo thứ tự xáo
k <- ceil(50 ms / median 5 lần đo
          V1 cutoff 0)
     (giới hạn k*n <= 2e7 tham chiếu)
for t in 0..29:
  x <- generate(d, n, seedFor(d,n,t))
  for c in shuffle(configs, seed):
    // ngoài giờ đo:
    tạo k bản sao x; System.gc()
    // trong giờ đo:
    t0 <- nanoTime(); sort k bản sao
    t1 <- nanoTime()
    // ngoài giờ đo:
    kiểm tra đã sắp xếp; cộng vào sink
    ghi (c, t, k, t1-t0, (t1-t0)/k)
```

### 4.5 Môi trường

Mọi lần đo được thực hiện trên cùng một máy với cấu hình trong Bảng 4.

| | |
|---|---|
| CPU | AMD Ryzen AI 7 H 350 (8 lõi, 16 luồng) |
| RAM | 32 GB |
| Hệ điều hành | Windows 11 Home 10.0.26300 |
| JDK | Oracle Java SE 25.0.2 LTS (HotSpot) |
| Cờ JVM | `-Xms4g -Xmx4g -XX:+UseSerialGC -XX:+AlwaysPreTouch` |
| Ghim lõi | mặt nạ `0xFF` (CPU logic 0–7) |
| Chế độ nguồn | `Best Performance` |

Bảng 4: Môi trường thí nghiệm

### 4.6 Hiệu chỉnh phép đo (chạy thử)

Lần chạy thử (5 trial, RANDOM, $n \in \{10^3, 10^5, 10^6\}$, cutoff 0–40) và phép thử ghim lõi cho các kết luận:

- **K2 (độ nhiễu):** CV trung vị giữa các trial là 1,0–2,0%.
- **K4 (GC):** mọi lần GC đều do `System.gc()` gọi chủ động, không có GC trong vùng đo.
- **K5 (loại lõi):** cùng cấu hình $n = 10^5$, chạy trên CPU logic 8–15 chậm hơn CPU 0–7 khoảng 47% (V1: 22,9 so với 15,6 ms) và 58% (V2: 17,8 so với 11,3 ms), cho thấy máy có hai loại lõi (Zen 5 và Zen 5c). Nếu để hệ điều hành tự chuyển luồng giữa hai loại lõi, nhiễu sẽ rất lớn, nên JVM được ghim vào CPU 0–7.
- Đường cong của V1 vẫn đang đi xuống ở cutoff 40, nên khoảng cutoff được mở rộng đến 80 (bước 4 sau 40 để giữ thời gian chạy hợp lý).

Lần chạy thật (TN1 và TN2) mất khoảng 37 phút.

### 4.7 Phân tích thống kê và quy tắc ra quyết định

Các quy tắc được ghi trong `analyze.py` trước khi phân tích:

- **R1:** thước đo chính là tỉ số ghép cặp $r = t(\text{cutoff})/t_{\text{base}}$ trong cùng một trial, với $t_{\text{base}}$ là trung bình của cutoff 0 và 1, để loại bỏ dao động giữa các trial.
- **R2:** $L^*$ là cutoff có median của $r$ nhỏ nhất.
- **R3:** vùng gần tối ưu gồm các cutoff có $\text{median}(r) \le \text{median}(r_{L^*}) \times 1{,}01$ (ngưỡng 1% chọn theo mức nhiễu nền của A/A test).
- **R4:** so sánh 7 và 10 bằng kiểm định Wilcoxon ghép cặp (**?**) trên 30 trial, $\alpha = 0{,}05$, hiệu chỉnh Holm (**?**) trong họ 3 so sánh {7 vs 10, $L^*$ vs 7, $L^*$ vs 10}.

Khoảng tin cậy 95% của median được tính bằng bootstrap (2000 lần lặp). Ngoài ra, để có một giá trị dùng chung cho mọi $n$, ta tính *regret* của mỗi cutoff (mức chậm hơn so với $L^*$ của từng $n$) và chọn cutoff có regret lớn nhất nhỏ nhất (phân tích bổ sung sau khi có kết quả).

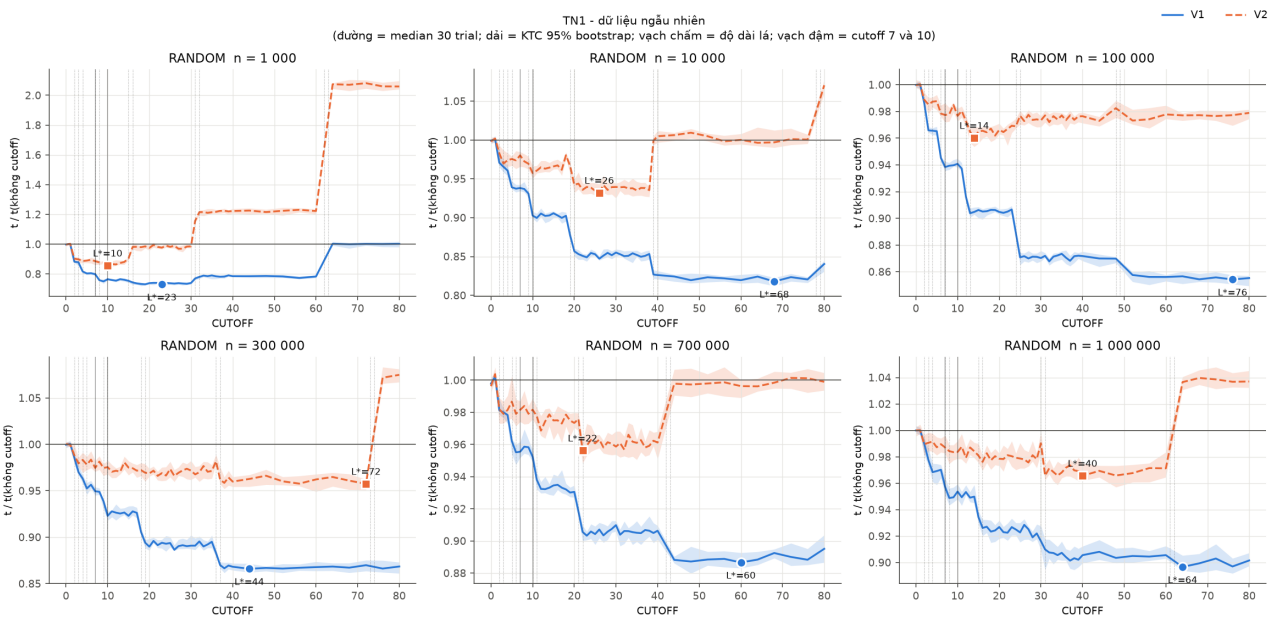
## 5 Kết quả

### 5.1 Chất lượng phép đo

Trên lần chạy thật: **A/A test** (cutoff 1 so với cutoff 0, cùng thuật toán): chênh lệch ghép cặp có median từ −0,39% đến +0,76% ở TN1 và TN2 (trừ V2 SORTED: +1,30%), khoảng tứ phân vị 1,1–3,4% (V2 SORTED: 4,6%). **Độ nhiễu:** CV trung vị theo cấu hình là 0,7–3,1% (V2 SORTED: 12,7%; TN3: khoảng 11,5%). **Hiệu ứng thứ tự:** tương quan Spearman giữa vị trí chạy trong trial và sai số chuẩn hoá có $|\rho| < 0{,}07$ ở mọi trường hợp. **Trôi:** median 5 trial đầu và 5 trial cuối chênh nhau tối đa 3,5% ($n = 700\,000$) ở TN1 và TN2. **GC:** mỗi GC log có đúng 3065 lần dừng, bằng số lần gọi `System.gc()` (102 cấu hình × 30 trial +5), tức không có GC nào trong vùng đo. **Độ dài lô:** $k$ thay đổi từ 498 ($n = 10^3$) đến 1 ($n \ge 3 \cdot 10^5$); mỗi lô dài 9–222 ms, riêng V2 SORTED chỉ 1,8 ms, giải thích vì sao trường hợp này nhiễu nhất.

### 5.2 TN1: dữ liệu ngẫu nhiên

Hình 1 xác nhận hiệu ứng bậc thang: mọi bước nhảy của đường cong đều nằm tại một độ dài lá, giữa hai độ dài lá đường cong nằm ngang (chỉ còn nhiễu). Vì thế giá trị $L^*$ cụ thể trong Bảng 5 chỉ có nghĩa đến mức lớp lá: chẳng hạn ở $n = 100\,000$, V1 có $L^* = 76$ nhưng mọi cutoff 49–96 đều là cùng một thuật toán (lá $\{48, 49\}$), nên "76" không tốt hơn 52 hay 80.




Hình 1: TN1, dữ liệu ngẫu nhiên: tỉ số $r = t/t(\text{không cutoff})$ theo CUTOFF (đường liền: V1, đường đứt: V2; dải mờ: KTC 95% bootstrap của median; vạch chấm: các độ dài lá; vạch đậm: cutoff 7 và 10). Đường cong là hàm bậc thang, các bậc nằm đúng tại độ dài lá. V1 tiếp tục giảm đến vùng 40–80; V2 đạt đáy ở vùng 10–40 rồi tăng mạnh khi lá quá dài.

| | V1 | | | | V2 | | | |
|---|---|---|---|---|---|---|---|---|
| $n$ | $L^*$ | lớp lá | vùng gần tối ưu | tăng tốc | $L^*$ | lớp lá | vùng gần tối ưu | tăng tốc |
| 1 000 | 23 | {15, 16} | 17–29 (không liền) | 27,0% | 10 | {7, 8} | 10, 12 | 14,4% |
| 10 000 | 68 | {39, 40} | 44–76 | 18,2% | 26 | {19, 20} | 22–38 | 6,8% |
| 100 000 | 76 | {48, 49} | 52–80$^\dagger$ | 14,6% | 14 | {12, 13} | 13–24 | 4,0% |
| 300 000 | 44 | {36, 37} | 37–80$^\dagger$ | 13,4% | 72 | {36, 37} | 22–26, 37–72 | 4,3% |
| 700 000 | 60 | {42, 43} | 44–80$^\dagger$ | 11,3% | 22 | {21, 22} | 22–40 | 4,4% |
| 1 000 000 | 64 | {61, 62} | 34–80$^\dagger$ | 10,3% | 40 | {30, 31} | 31–60 | 3,4% |

Bảng 5: $L^*$, lớp lá, vùng gần tối ưu (trong 1% so với $L^*$) và mức tăng tốc so với không cutoff (TN1). Dấu †: vùng gần tối ưu chạm biên 80 của khoảng đo.

Với **V1**, lá tối ưu dài 36–62 phần tử khi $n \geq 10^4$ và 15–16 khi $n = 10^3$; mức tăng tốc tại $L^*$ là 10,3–27,0%, trong khi cutoff 7 và 10 chỉ đạt 4,3–23,5%. Ở bốn kích thước lớn nhất, vùng gần tối ưu chạm biên 80, tức tối ưu thật có thể còn nằm xa hơn. Với **V2**, lá tối ưu dài 7–37 phần tử và mức tăng tốc nhỏ hơn nhiều (3,4–14,4%), vì V2 không có bước sao chép nên phần chi phí mà cutoff loại bỏ được cũng nhỏ hơn. Đường cong V2 tăng vọt khi lá quá dài: ở $n = 10^3$, lá {62, 63} làm V2 chậm gấp 2,07 lần so với không cutoff.

Về thời gian tuyệt đối, V2 nhanh hơn V1 30–44% ở mọi $n$ khi không cutoff (ví dụ $n = 10^6$: 171,5 so với 249,1 ms). Một điểm bất thường: ở $n = 10^3$ với cutoff $\geq 63$, hai phiên bản làm đúng cùng các lá insertion sort và V2 có phần merge rẻ hơn, nhưng V2 lại chậm hơn V1 16% (45,0 so với 38,9 $\mu$s). Bước nhảy tương tự xuất hiện ở $n = 3 \cdot 10^5$, cutoff 80. Nhiều khả năng đây là hiệu ứng của trình biên dịch JIT (cách inline, bố trí mã) chứ không phải của thuật toán; điều này chưa được kiểm chứng (Mục 8).

**Một giá trị dùng chung cho mọi $n$** Bảng 6 xếp hạng cutoff theo regret lớn nhất trên sáu kích thước. Với V1, cutoff 24–29 không bao giờ chậm hơn tối ưu của từng $n$ quá 4,5% (tốt nhất: 26, tối đa 3,6%); với V2, cutoff 10–14 không chậm hơn quá 3,5% (tốt nhất: 10, tối đa 2,7%).

| Bản | CUTOFF | Regret lớn nhất | TB nhân $r$ |
|---|---|---|---|
| V1 | 26 (tốt nhất) | 3,6% | 0,860 |
| | 24–29 | ≤ 4,5% | 0,860–0,865 |
| | 10 | 10,4% | 0,904 |
| | 7 | 14,7% | 0,921 |
| V2 | 10 (tốt nhất) | 2,7% | 0,954 |
| | 9–14 | ≤ 4,0% | 0,953–0,961 |
| | 7 | 5,2% | 0,964 |

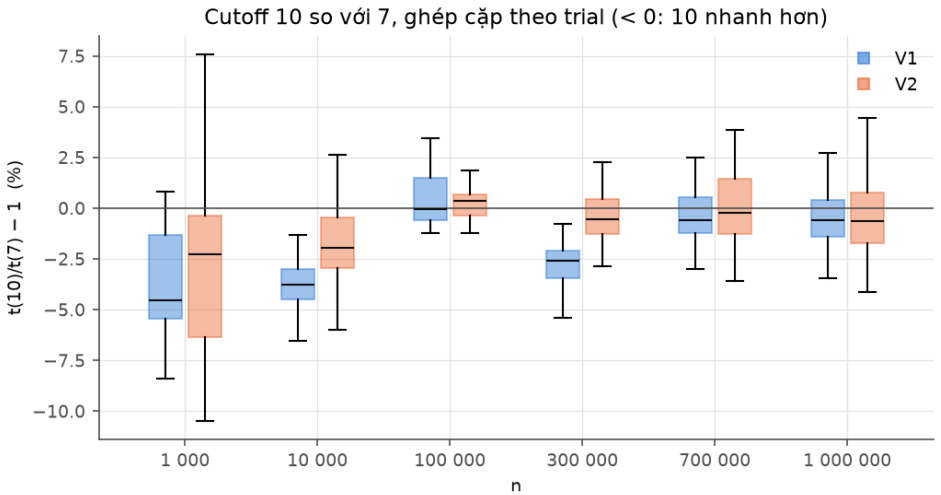
Bảng 6: Regret lớn nhất trên sáu kích thước của TN1 (mức chậm hơn so với $L^*$ của từng $n$) và trung bình nhân của $r$

### 5.3 Cutoff 7 hay 10?

Trả lời RQ2: 10 nhanh hơn 7 có ý nghĩa thống kê ở 5/12 trường hợp (chênh 2,0–4,7%), các trường hợp còn lại không khác biệt có ý nghĩa, và không trường hợp nào 7 nhanh hơn 10. Median chênh lệch dương ở 11/12 trường hợp (Hình 2). Vậy trên máy này 10 tốt hơn hoặc bằng 7, nhưng cả hai đều kém xa vùng tối ưu (Bảng 7, Bảng 6).

| $n$ | Bản | $t_7/t_{10} - 1$ | $p_{\text{Holm}}$ | Cùng lớp lá? | Kết luận |
|---|---|---|---|---|---|
| 1 000 | V1 | +4,73% | $5{,}2 \cdot 10^{-8}$ | không | 10 nhanh hơn |
| | V2 | +2,30% | 0,046 | không | 10 nhanh hơn |
| 10 000 | V1 | +3,93% | $5{,}6 \cdot 10^{-9}$ | không | 10 nhanh hơn |
| | V2 | +1,98% | $7{,}1 \cdot 10^{-5}$ | không | 10 nhanh hơn |
| 100 000 | V1 | +0,05% | 0,31 | có | không khác biệt |
| | V2 | −0,35% | 0,35 | có | không khác biệt |
| 300 000 | V1 | +2,68% | $1{,}3 \cdot 10^{-8}$ | không | 10 nhanh hơn |
| | V2 | +0,56% | 0,19 | không | không khác biệt |
| 700 000 | V1 | +0,59% | 0,12 | không | không khác biệt |
| | V2 | +0,23% | 0,84 | không | không khác biệt |
| 1 000 000 | V1 | +0,58% | 0,19 | không | không khác biệt |
| | V2 | +0,66% | 0,12 | không | không khác biệt |

Bảng 7: Kiểm định Wilcoxon ghép cặp: cutoff 7 so với 10 (TN1)



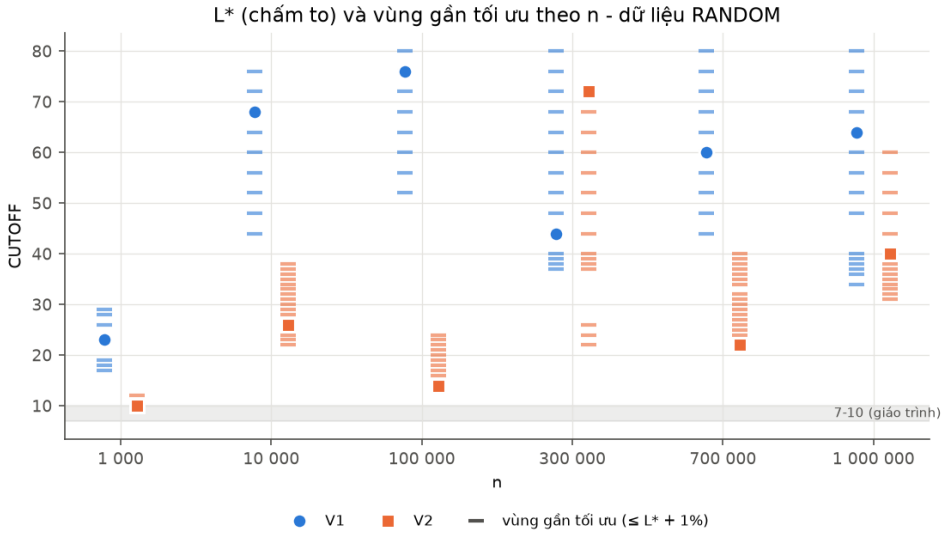

Hình 2: Chênh lệch ghép cặp $t(10)/t(7) - 1$ theo trial (hộp: tứ phân vị; giá trị âm: 10 nhanh hơn). Ở $n \le 10^4$ và $n = 3 \cdot 10^5$ (V1), 10 nhanh hơn rõ; ở $n = 10^5$, 7 và 10 cùng lớp lá nên hộp nằm quanh 0.

Các trường hợp 7 và 10 cùng lớp lá (ở $n = 10^5$, cả hai đều cho lá $\{6, 7\}$) là *đối chứng âm*: đây là cùng một thuật

toán nên kiểm định không được phép tìm thấy khác biệt. Trong 12 trường hợp cùng lớp lá ở TN1, TN2 và TN3, có 11 trường hợp đúng là không khác biệt; trường hợp còn lại (TN2 SORTED, V1: $p_{\text{Holm}} = 0{,}045$, "7 nhanh hơn 0,35%") là một *dương tính giả*, đúng với tỉ lệ khoảng 5% kỳ vọng khi $\alpha = 0{,}05$. Kết luận này vì vậy phải được loại bỏ.

### 5.4 $L^*$ theo kích thước $n$

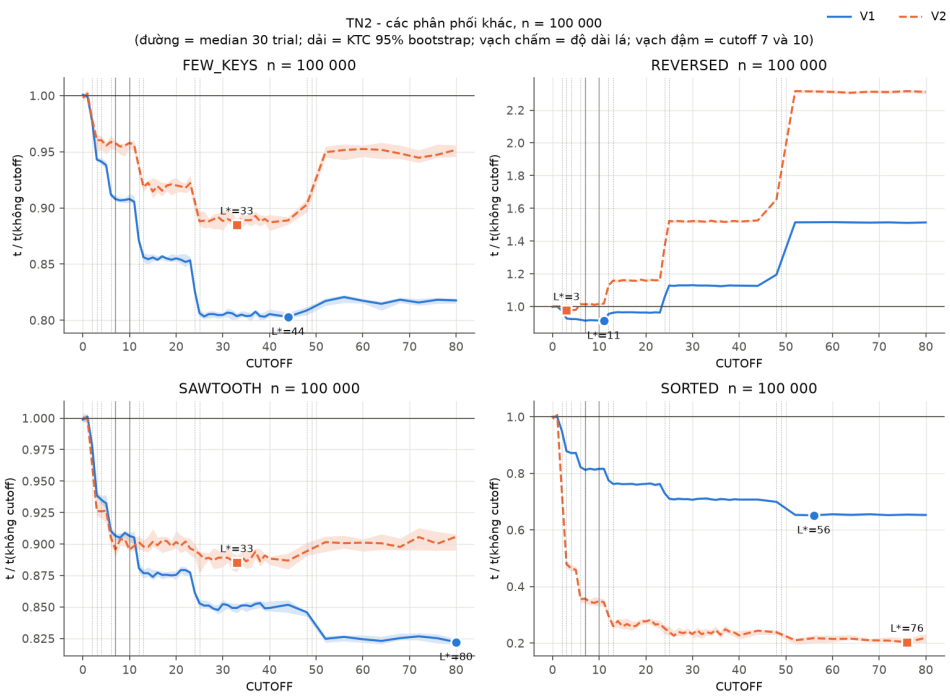
$L^*$ của V1 lần lượt là 23, 68, 76, 44, 60, 64 và của V2 là 10, 26, 14, 72, 22, 40 khi $n$ tăng từ $10^3$ đến $10^6$ (Hình 3). Các con số này dao động mạnh, nhưng phần lớn dao động là do cách chọn argmin trong một vùng gần như phẳng và do lớp lá thay đổi theo $n$: vùng gần tối ưu của các $n$ chồng lên nhau rất nhiều (V1: khoảng 37–80 với mọi $n \geq 10^4$; V2: khoảng 22–38 với $n \in \{10^4, 7 \cdot 10^5, 10^6\}$). Khác biệt thật duy nhất là $n = 10^3$, nơi cả hai phiên bản có lá tối ưu ngắn hơn rõ rệt (V1: 15–16, V2: 7–8).




Hình 3: $L^*$ (chấm to) và các cutoff thuộc vùng gần tối ưu (vạch ngang) theo $n$, dữ liệu RANDOM. Dải xám là khoảng 7–10 của giáo trình.

### 5.5 TN2: các phân phối khác

Với $n = 100\,000$ (Hình 4): **REVERSED**: lá tối ưu ngắn nhất, V1 có $L^* = 11$ (lá $\{6, 7\}$, tăng tốc 8,7%), V2 có $L^* = 3$ (lá $\{2, 3\}$, 2,4%); với V2 cutoff 7 và 10 còn chậm hơn không cutoff 1,4–1,7%. Đây là trường hợp xấu nhất của insertion sort ($\sim L^2/2$ thay vì $L^2/4$, tức $a$ tăng gấp đôi). **SORTED**: lá tối ưu dài nhất (lá $\{48, 49\}$ cho cả hai phiên bản), tăng tốc 34,8% (V1) và 79,6% (V2), vì insertion sort trên dữ liệu đã có thứ tự chỉ tốn $L - 1$ phép so sánh; với V2, mọi merge đều bị bỏ qua. **FEW_KEYS**: lá $\{24, 25\}$ cho cả hai phiên bản, tăng tốc 19,7% (V1) và 11,5% (V2). **SAWTOOTH**: V1 có $L^* = 80$, chạm biên khoảng đo (lá $\{48, 49\}$, 17,8%); V2 có lá $\{24, 25\}$ (11,5%). Ở cả bốn phân phối, 7 và 10 cùng lớp lá nên không thể phân biệt được.

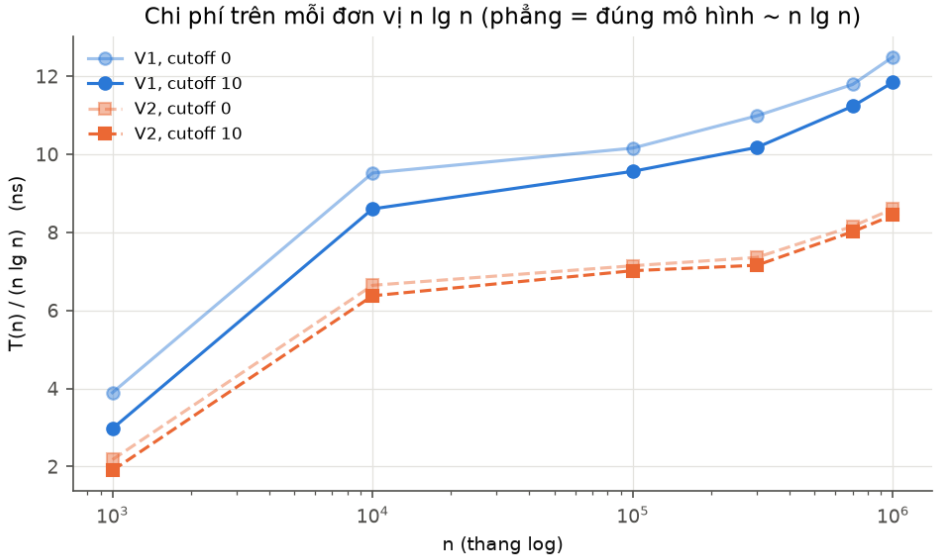



Hình 4: TN2, $n = 100\,000$: tỉ số $r$ theo CUTOFF cho FEW_KEYS, REVERSED, SAWTOOTH và SORTED. REVERSED cho lá tối ưu ngắn nhất, SORTED cho lá dài nhất.

Dữ liệu REVERSED được sắp xếp nhanh hơn RANDOM nhiều (V1 không cutoff: 4,9 so với 16,9 ms). Lý do: khi trộn hai nửa của dãy giảm dần, mọi phần tử nửa phải nhỏ hơn nửa trái, nên chỉ cần $\sim m/2$ phép so sánh thay vì $\sim m$, nhánh rẽ luôn đoán đúng, và các đối tượng `Double` được cấp phát theo thứ tự giá trị nên truy cập bộ nhớ gần như tuần tự. Với RANDOM, nhánh rẽ trong merge đoán sai khoảng một nửa số lần và các đối tượng nằm rải rác trên heap.

### 5.6 Kiểm tra mô hình $N \lg N$

Hồi quy $\lg T = b \lg N + c$ trên sáu kích thước của TN1 cho $b = 1{,}244$ (V1, không cutoff), 1,272 (V1, cutoff 10), 1,271 (V1, $L^*$ của từng $n$), 1,267, 1,282, 1,280 (V2, cùng thứ tự). Với $N \lg N$ trong khoảng $10^3$–$10^6$, hệ số mũ kỳ vọng chỉ khoảng 1,10. Hệ số đo được lớn hơn đáng kể, và Hình 5 cho thấy $T/(n \lg n)$ tăng theo $n$: số phép so sánh vẫn theo $N \lg N$, nhưng chi phí mỗi phép so sánh tăng khi dữ liệu vượt khỏi các tầng cache (Mục 6.2).

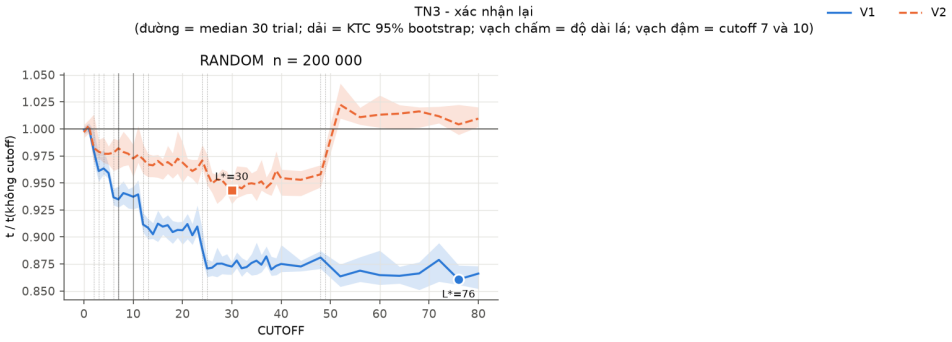



Hình 5: Thời gian chia cho $n \lg n$ theo $n$ (trục log). Nếu chi phí đúng là $\sim c\,n \lg n$ thì đường nằm ngang; ở đây đường tăng dần theo $n$.

### 5.7 TN3: xác nhận trên $n = 200\,000$

**Dự đoán.** $n = 200\,000$ có cùng tập độ dài lá với $n = 100\,000$ (Bảng 1), nên đường cong phải có bậc ở cùng vị trí và lớp lá tối ưu phải như ở $n = 100\,000$: V1 lá $\{48, 49\}$, V2 trong vùng 13–24; 7 và 10 cùng lớp lá nên không khác biệt.

**Kết quả.** V1: $L^* = 76$, lá $\{48, 49\}$, tăng tốc 13,9%, vùng gần tối ưu 52–80 (chạm biên). V2: $L^* = 30$, lá $\{24, 25\}$, tăng tốc 5,7%, vùng gần tối ưu 26–38. 7 so với 10: không khác biệt ($p_{\text{Holm}} = 0{,}52$ và 0,54) (Hình 6).




Hình 6: TN3, RANDOM, $n = 200\,000$: tỉ số $r$ theo CUTOFF. Các bậc nằm đúng ở các độ dài lá như $n = 100\,000$.

**Đánh giá.** Vị trí các bậc, lớp lá tối ưu của V1 và kết luận "7 = 10" khớp dự đoán. Lớp lá tối ưu của V2 dài hơn một bậc so với $n = 10^5$ ($\{24, 25\}$ thay vì $\{12, 13\}$), nhưng ở $n = 10^5$ hai lớp này đã chênh nhau dưới 1%, nên đây là kết quả nằm trong vùng phẳng. Lưu ý TN3 được chạy tay vào ngày khác, không qua script (không có GC log, không ghi nhận mặt nạ ghim lõi), CV khoảng 11,5% và thời gian trôi −15,6% từ 5 trial đầu đến 5 trial cuối, nên TN3 chỉ là xác nhận yếu; nhờ thiết kế ghép cặp, tỉ số $r$ vẫn dùng được.

## 6 Thảo luận

### 6.1 Đối chiếu giả thuyết

Bảng 8 tổng hợp kết luận cho năm giả thuyết đã nêu ở Bảng 3.

| | Kết luận | Bằng chứng |
|---|---|---|
| H1 | Đúng một phần | V1: 10,3–27,0% (đúng hoặc vượt); V2: 3,4–14,4%, 5/6 kích thước dưới 10% |
| H2 | Bác bỏ | Chỉ 2/12 trường hợp (V2, $n = 10^3$: 10; $n = 10^5$: 14); V1 luôn $\geq 23$ |
| H3 | Bác bỏ (dạng mạnh) | $L^*$ V1: 23–76, V2: 10–72; nhưng vùng gần tối ưu chồng lấn nhiều khi $n \geq 10^4$, khác biệt rõ chỉ ở $n = 10^3$ |
| H4 | Ủng hộ | Vùng trong 1% trải rộng: V1 từ 44–76 đến 34–80 ($n \geq 10^4$), V2 từ 13–24 đến 22–72; hẹp chỉ ở $n = 10^3$ (V2: 10–12) |
| H5 | Ủng hộ | 5/6 kích thước $L^*$(V2) nhỏ hơn hẳn; $n = 3 \cdot 10^5$ hoà (cùng lá $\{36, 37\}$); cutoff chung: 10 so với 26 |

Bảng 8: Kết luận về các giả thuyết

### 6.2 Giải thích

**Vì sao $L^*$ của V1 lớn hơn nhiều so với 7–10?** Theo (1), $L^* \propto b/a$. Mỗi tầng merge của V1 chép toàn bộ đoạn sang `aux` rồi ghi lại, mỗi phép so sánh gọi `compareTo` của `Double` qua con trỏ (đối tượng nằm rải rác trên heap) và nhánh rẽ đoán sai khoảng một nửa số lần, nên $b$ lớn. Trong khi đó insertion sort trên một lá ngắn chỉ chạm vào vài chục tham chiếu đã nằm trong L1 và nhánh rẽ dễ đoán, nên $a$ nhỏ.

**Vì sao $L^*$(V2) < $L^*$(V1)?** V2 bỏ bước sao chép, làm $b$ giảm rõ rệt (V2 nhanh hơn V1 30–44%) trong khi $a$ không đổi (cùng mã insertion sort), nên $L^*$ giảm. `MergeX` và Java 6 dùng 7 cho chính phiên bản có đủ cải tiến

như V2; với V2, kết quả 10 của tôi khá gần giá trị đó.

**Vì sao $n = 10^3$ khác?** Với $n = 10^3$, mỗi lô có $k = 498$ bản sao dùng chung 1000 đối tượng `Double`; toàn bộ dữ liệu luôn nằm trong cache L1/L2 nên mỗi tầng merge rất rẻ ($b$ nhỏ), kéo $L^*$ xuống. Với $n$ lớn, mỗi tầng merge phải duyệt cả mảng trong RAM hoặc L3 nên $b$ lớn hơn, còn insertion sort trên lá vẫn chạy trong L1. Điều này cũng giải thích vì sao mô hình dự đoán $L^*$ không phụ thuộc $N$ nhưng thực tế lại phụ thuộc: $b$ chính là một hàm của $N$ qua phân cấp bộ nhớ.

**Vì sao REVERSED cho $L^*$ nhỏ nhất, SORTED lớn nhất?** REVERSED là trường hợp xấu nhất của insertion sort ($a$ gấp đôi) nhưng lại là trường hợp merge rẻ (ít so sánh, nhánh dễ đoán), nên $b/a$ nhỏ. SORTED là trường hợp tốt nhất của insertion sort (chi phí tuyến tính, gần như $a \to 0$), nên lá càng dài càng tốt.

**Vì sao hệ số mũ log-log $\approx 1{,}25$ lớn hơn 1,10?** Số phép so sánh vẫn tăng theo $N \lg N$, nhưng chi phí mỗi phép tăng theo $n$: với $n = 10^6$, các đối tượng `Double` (khoảng 16 MB) cùng hai mảng tham chiếu vượt quá cache L3 nên phần lớn phép so sánh phải chờ RAM, trong khi với $n = 10^3$ mọi thứ nằm trong L1.

**Bài học thống kê.** Với hơn 60 kiểm định, vài dương tính giả là điều tất yếu; trường hợp TN2 SORTED V1 cho thấy đối chứng âm (cùng lớp lá) giúp phát hiện chúng. Ngoài ra, $L^*$ được chọn là argmin trên chính dữ liệu dùng để kiểm định "$L^*$ vs 7", nên p-value của các so sánh này lạc quan hơn thực tế (lời nguyền người thắng cuộc); cần một lần đo xác nhận độc lập.

### 6.3 So sánh với giáo trình và khuyến nghị

Giáo trình nêu 7 (Java 6, `MergeX`) và khoảng 10, đồng thời cho rằng giá trị trong khoảng 5–15 đều tốt. Với V2 (cài đặt gần `MergeX` nhất), kết quả của tôi phù hợp: cutoff 10 là lựa chọn tốt nhất theo tiêu chí regret, chậm hơn tối ưu từng $n$ tối đa 2,7%. Với V1 (chỉ có cutoff), khoảng 5–15 là quá nhỏ: cutoff 26 nhanh hơn cutoff 10 khoảng 2,6–7,3% tùy $n$. Khác biệt đến từ việc V1 còn bước sao chép (làm $b$ lớn), phần tử là `Double` đóng hộp so sánh qua con trỏ, và CPU hiện đại đoán nhánh tốt cho insertion sort.

Khuyến nghị cho máy này: **V1 dùng CUTOFF trong khoảng 24–29 (chọn 26); V2 dùng 10–14 (chọn 10)**. Nên báo cáo CUTOFF như một vùng chứ không phải một số, vì (i) đường cong phẳng quanh tối ưu, (ii) các cutoff cùng lớp lá là cùng một thuật toán, và (iii) lớp lá tối ưu dịch chuyển theo $n$, nên một con số "tối ưu" cho một $n$ không mang sang $n$ khác được.

## 7 Kết luận

Trên máy AMD Ryzen AI 7 H 350 với Java 25 và phần tử `Double`, cutoff sang insertion sort luôn có lợi trên dữ liệu ngẫu nhiên: tối đa 10,3–27,0% với V1 và 3,4–14,4% với V2 (RQ1). Giá trị dùng chung tốt nhất là CUTOFF $\approx 26$ cho V1 (chậm hơn tối ưu từng $n$ tối đa 3,6%) và CUTOFF = 10 cho V2 (tối đa 2,7%). Giữa 7 và 10, 10 nhanh hơn có ý nghĩa ở 5/12 trường hợp và chưa bao giờ chậm hơn (RQ2). $L^*$ phụ thuộc vào phiên bản (bỏ bước sao chép làm $L^*$ giảm mạnh), vào phân phối (REVERSED: lá 2–7; SORTED: lá $\geq 48$) và vào $n$ chủ yếu qua cấu trúc lá và trạng thái cache (RQ3). Bài học về phương pháp: vì mergesort chia đôi, CUTOFF chỉ có tác dụng thông qua độ dài lá, nên phải phân tích theo lớp lá, báo cáo kết quả dưới dạng vùng, và dùng các cutoff cùng lớp lá làm đối chứng âm. Hướng mở rộng: đo với kiểu nguyên thuỷ và trên máy khác, dùng JMH, thử insertion sort không dùng `exch`, mở rộng khoảng cutoff vượt 80 và chạy một lần xác nhận độc lập với seed mới cho các giá trị đã chọn.

## 8 Hạn chế và mối đe dọa đến tính hợp lệ

**Tính hợp lệ bên ngoài.** Chỉ đo trên một máy, một JDK, một kiểu phần tử (`Double` đóng hộp); với kiểu nguyên thuỷ hoặc CPU khác, $L^*$ có thể khác hẳn. Insertion sort dùng `exch` như slide; biến thể dịch phần tử (half-exchange) sẽ làm $a$ giảm và $L^*$ tăng.

**Tính hợp lệ bên trong.** (1) CUTOFF là tham số lúc chạy, JIT không gập được thành hằng số như khi khai báo `static final`; điều này giữ cho mọi cutoff cùng một mã máy nhưng có thể khác với cài đặt thực tế. (2) $n$ nhỏ được đo ở trạng thái cache nóng ($k$ bản sao dùng chung đối tượng), $n$ lớn ở trạng thái cache lạnh. (3) $k$ được hiệu chỉnh từ 5 lần đo đơn lẻ (chậm hơn trạng thái ổn định), nên nhiều lô ngắn hơn mục tiêu 50 ms; riêng V2 SORTED lô chỉ 1,8 ms và CV 12,7%. (4) Hiện tượng V2 chậm hơn V1 ở lá dài (Mục 5.2) chưa được giải thích, nghi do JIT. (5) Output của JVM (giá trị $k$, thời gian warm-up) không được ghi vào `run.log`. (6) `SortTest` chỉ kiểm tra cutoff $\leq 40$; cutoff 44–80 chỉ được kiểm tra bằng phép thử đã-sắp-xếp trong harness.

**Thiết kế và thống kê.** (7) Vùng gần tối ưu của V1 chạm biên 80 ở 4/6 kích thước TN1, ở SAWTOOTH, SORTED và TN3; tối ưu thật có thể lớn hơn 80. (8) Lưới bước 4 sau 40 đo lặp nhiều cutoff cùng lớp lá; lưới theo biên lớp lá sẽ hiệu quả hơn. (9) $L^*$ được chọn bằng argmin theo từng cutoff thay vì theo lớp lá, và được kiểm định trên cùng dữ liệu (lời nguyền người thắng cuộc). (10) Nhiều kiểm định dẫn đến dương tính giả (đã thấy một trường hợp). (11) TN3 chạy trong điều kiện khác TN1 (không qua script, nhiễu và trôi lớn), và $n = 200\,000$ không có trong manifest (seed vẫn tái lập được bằng `seedFor`).

## A Tái lập thí nghiệm

```
javac -encoding UTF-8 -d out src/sort/*.java src/test/*.java src/bench/*.java
java -cp out test.SortTest
java -cp out bench.DataGenerator
powershell -ExecutionPolicy Bypass -File scripts\run_all.ps1 -Pilot
powershell -ExecutionPolicy Bypass -File scripts\run_all.ps1 -AffinityMask 0xFF -
    Cutoffs "0-40,44-80:4"
python analysis/pilot_check.py results/raw/20261108_0403_pilot results/raw/affinity
python analysis/analyze.py results/raw/20261108_0426 results/raw/TN3
```

Lần chạy thật: `results/raw/20261108_0426` (TN1, TN2) và `results/raw/TN3`.

## B Sử dụng AI

| Thành phần | Người thực hiện / vai trò của AI |
|---|---|
| InsertionSort, MergeCutOff, MergeOptimized | Sinh viên tự viết, có AI hỗ trợ kiểm tra tính đúng sai và viết gọn code |
| SortTest | Sinh viên viết, có AI hỗ trợ lên ý tưởng các trường hợp cần kiểm thử và khung kiểm thử |
| DataGenerator, Experiment, run_all.ps1 | AI viết |
| pilot_check.py, analyze.py | AI viết |
| Thiết kế thí nghiệm, quyết định | Sinh viên đưa ra đề xuất và AI đưa ra nhận xét và gợi ý các hướng thí nghiệm và quyết định tối ưu hơn |
| Phân tích, giải thích, báo cáo | Sinh viên viết nội dung về đánh giá và phân tích từ biểu đồ, từ kết quả thực nghiệm dưới dạng văn bản chữ, AI hỗ trợ cách trình bày, khung báo cáo và nội dung cần có |

Bảng 9: Phân công giữa sinh viên và AI